

PRACTICA 3 SEMAFOROS

Código a reparar

```
/*
Para compilar incluir la librería m (matemáticas)
Ejemplo:
gcc -o mercator mercator.c -lm
*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <math.h>
#include <sys/time.h>
#include <sys/wait.h>
#include <sys/shm.h>
#include <semaphore.h>

#define NPROCS 4
#define SERIES_MEMBER_COUNT 200000

typedef struct {
    double sums[NPROCS];
    int proc_count;
    int start_all;
    double x_val;
    double res;
} SHARED;

SHARED *shared;

double get_member(int n, double x) {
    int i;
    double numerator = 1;
    for(i=0; i<n; i++ )
```

```

        numerator = numerator*x;
    if (n % 2 == 0)
        return ( - numerator / n );
    else
        return numerator/n;
}

void proc(int proc_num) {
    int i;

    // Espera a que master_proc ponga la bandera en 1 para iniciar
    while(!(shared->start_all));

    // Cada proceso realiza el cálculo de los términos que le tocan
    shared->sums[proc_num] = 0;

    for(i=proc_num; i<SERIES_MEMBER_COUNT;i+=NPROCS)
        shared->sums[proc_num] += get_member(i+1, shared->x_val);
    // Incrementa la variable proc_count que es la cantidad de procesos
    shared->proc_count++;
    exit(0);
}

void master_proc(){
    int i;
    // Obtener el valor de x desde el archivo entrada.txt
    FILE *fp = fopen("entrada.txt","r");
    if(fp==NULL)
        exit(1);

    fscanf(fp,"%lf",&shared->x_val);
    fclose(fp);
    // el master_proc pone una bandera para que los procesos inicien
    shared->start_all=1;

    // Espera a que todos los procesos terminen su cálculo
    while (shared->proc_count != NPROCS) {}
}

```

```

    // Una vez que todos terminan, suma el total de cada uno
    shared->res = 0;
    for(i=0; i<NPROCS; i++)
        shared->res += shared->sums[i];
    exit(0);
}

//necesitamos dos semaforos, uno para la espera del proceso mae:

int main(){
    int *threadIdPtr;
    long long start_ts;
    long long stop_ts;
    long long elapsed_time;
    long lElapsedTime;
    struct timeval ts;
    int i;
    int p;
    int shmid;
    int status;

    // Solicita y conecta la memoria compartida
    shmid = shmget(0x1234, sizeof(SHARED), 0666|IPC_CREAT);
    shared = shmat(shmid, NULL, 0);

    // inicializa las variables en memoria compartida
    shared->proc_count = 0;
    shared->start_all = 0;
    gettimeofday(&ts, NULL);
    start_ts = ts.tv_sec; // Tiempo inicial
    for(i=0; i<NPROCS;i++) {
        p = fork();
        if(p==0)
            proc(i);
    }
}

```

```

p = fork();
if(p==0)
    master_proc();

printf("El recuento de ln(1 + x) miembros de la serie de Me
%d\n", SERIES_MEMBER_COUNT);
for(int i=0;i<NPROCS+1;i++) {
    wait(&status);
    if(status==0x100) // Si el master_proc termina con error
    {
        fprintf(stderr, "Proceso no puede abrir el archivo de
        break;
    }
}
gettimeofday(&ts, NULL);
stop_ts = ts.tv_sec; // Tiempo final
elapsed_time = stop_ts - start_ts;

printf("Tiempo = %lld segundos\n", elapsed_time);
printf("El resultado es %10.8f\n", shared->res);
printf("Llamando a la función ln(1 + %f) = %10.8f\n", shared->
log(1+shared->x_val));

// Desconecta y elimina la memoria compartida
shmdt(shared);
shmctl(shmid, IPC_RMID, NULL);
}

```

Código con semáforos

mercator.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <math.h>
#include <sys/wait.h>
#include <sys/shm.h>
#include <semaphore.h>
#include <fcntl.h>

#define NPROCS 4
#define SERIES_MEMBER_COUNT 200000

typedef struct {
    double sums[NPROCS];
    double x_val;
    double res;
} SHARED;

SHARED *shared;

sem_t *sem_master, *sem_prod;

double get_member(int n, double x) {
    int i;
    double numerator = 1;
    for (i = 0; i < n; i++)
        numerator = numerator * x;
    if (n % 2 == 0)
        return (-numerator / n);
    else
        return (numerator / n);
}

void proc(int proc_num) {

```

```

printf("Starting production process #%d\n", proc_num);
int i;

// Espera a la señal del maestro para empezar
sem_wait(sem_master);
printf("Signal recieved\n");

// Cada proceso realiza su cálculo
shared->sums[proc_num] = 0;
for (i = proc_num; i < SERIES_MEMBER_COUNT; i += NPROCS) {
    shared->sums[proc_num] += get_member(i + 1, shared->x_val);
}

// Señala al maestro que ha terminado
printf("Sending a signal to the master process\n");
sem_post(sem_prod);
printf("Exit process %d\n\n", proc_num);
exit(0);
}

void master_proc() {
    int i;
    printf("Starting master process\n");
    // Leer valor de x desde el archivo
    FILE *fp = fopen("entrada.txt", "r");
    if (fp == NULL)
        exit(1);

    fscanf(fp, "%lf", &shared->x_val);
    fclose(fp);

    // Señala a los hijos para que comiencen
    for (i = 0; i < NPROCS; i++) {
        printf("Allowing 1+\n");
        sem_post(sem_master); // Desbloquea a todos los hijos
    }
}

```

```

    // Espera a que todos los hijos terminen
    for (i = 0; i < NPROCS; i++) {
        printf("waiting production %d....\n", i);
        sem_wait(sem_prod);
    }

    // Suma los resultados finales
    shared->res = 0;
    for (i = 0; i < NPROCS; i++) {
        shared->res += shared->sums[i];
    }
    exit(0);
}

int main() {
    int i, p, shmid;
    int status;

    // Crear memoria compartida
    shmid = shmget(0x1234, sizeof(SHARED), 0666 | IPC_CREAT);
    shared = shmat(shmid, NULL, 0);

    sem_master = sem_open("/sem_master", O_CREAT, 0666, 0);
    sem_prod = sem_open("/sem_prod", O_CREAT, 0666, 0);

    for (i = 0; i < NPROCS; i++) {
        p = fork();
        if (p == 0)
            proc(i);
    }

    p = fork();
    if (p == 0)

```

```

        master_proc();

    for (int i = 0; i < NPROCS + 1; i++) {
        wait(&status);
    }

    printf("El resultado es %10.8f\n", shared->res);
    printf("ln(1 + %f) = %10.8f\n", shared->x_val, log(1 + shared->x_val));

    sem_close(sem_master);
    sem_close(sem_prod);
    sem_unlink("/sem_master");
    sem_unlink("/sem_prod");

    shmdt(shared);
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}

```

Cambios en el código

- Se quitaron todos los whiles que hacían la espera ocupada
- Se quito de la estructura el atributo `start_all` pues era una bandera que se substituye con los semáforos
- Se implementaron dos semáforos, uno para el proceso maestro y uno para cuando los otros procesos productores terminan
 - Se uso un for para mandar 4 señales al semáforo que activa a los productores, pues como todos los procesos escriben en diferentes direcciones de memoria

- Se uso un for para esperar a los 4 otros procesos, pues en cuanto termine uno volverá al mismo ciclo hasta que sean 4
- El semáforo `sem_master` es el que esperan los procesos productores, y el proceso maestro manda las señales a este semáforo
- El semáforo `sem_child_done` es el que manda las señales de regreso al proceso maestro

MAKEFILE

```
CC=gcc

all: start clean mercator.exe

mercator.exe: mercator.o
    $(CC) -o mercator.exe mercator.o -lm

mercator.o: mercator.c
    $(CC) -c mercator.c

clean:
    rm -f *.o *.exe

start:
    echo "Iniciando el proceso de compilacion"
```